
The Bottleneck Moves: Removing TCP's Loss Response on a Known-Lossy Link

Huacong Tang
University of Southern California

Xiao Yang
University of Southern California

Jiachen Lu
University of Southern California

 [Code](#)  [Demo Video](#)

Abstract

Random loss on a dedicated link is not congestion, but TCP cannot tell the difference. On a 100 Mbps path with 200 ms round-trip time and 20% loss in each direction it delivers 0.07 Mbps, three orders of magnitude below the link's erasure-channel capacity of 78 Mbps. From sender-side kernel telemetry over a 66-cell $\text{RTT} \times \text{loss}$ grid we attribute the collapse to two mechanisms the link does not justify: window reduction on loss, and exponential retransmission-timeout backoff. We remove each one and measure it separately. A twenty-line loadable congestion-control module that declines to read loss as congestion and holds the window at the bandwidth-delay product raises a complete, MD5-verified 1 GiB transfer to **10.75 Mbps**. It lifts **61 of 66** grid cells above 10 Mbps and matches stock cubic cell by cell on the lossless row. Which kernel hook does the pinning is worth a factor of two: `cong_avoid()` is never reached when the connection never leaves recovery. We expose backoff removal as a `sysctl` on a patched kernel, so both arms share one binary and one boot. It is visible in the timer but buys no throughput, because by then the bottleneck has moved above the transport: 20 of 24 samples are `app_limited`. A raw TCP stream with no application-level window reaches 37.8 Mbps on the same kernel.

1 Introduction

A leased, dedicated, lossy long-haul link is a common object: a satellite hop, a transoceanic circuit, a link into a partially connected region. Its loss belongs to the medium, not to anyone queueing behind anyone else. TCP was designed for the opposite world, and since Jacobson [1] reads a lost segment as evidence of congestion, responding twice: it reduces its window, and if a retransmission is itself lost it doubles its timeout [4]. Both responses are correct on a shared path and neither is earned here.

The cost is not marginal. On a 100 Mbps link with 200 ms round-trip time and 20% loss in each direction the erasure-channel capacity is $(1 - p)R \approx 78$ Mbps, and an ordinary `scp` over stock cubic delivers **0.07 Mbps**, 0.09% of it; at its 23 kbps steady state, moving a 1 GiB file would take four days. The Mathis law $B \approx 1.22 \text{ MSS} / (\text{RTT} \sqrt{p})$ [3] puts the ceiling for any window-halving sender at 0.16 Mbps here, which says the window and not the timer is the dominant term. This paper tests that by removing each mechanism in turn.

The diagnosis holds and the fix is small; the main finding is what is left over. Table 1 summarizes the paper: each row removes one layer of machinery above or inside TCP, and the capacity still missing at the top divides into roughly a factor of four above the transport and a factor of two inside it.

Table 1: The same link (100 Mbps, 200 ms RTT, 20% loss each way) and the same patched kernel, with progressively less machinery above TCP. The bound is the erasure-channel capacity $(1 - p)R$. The top row is our own paced-UDP protocol [7], measured on this testbed at MTU 9001, whose smaller header overhead raises the bound to 79.7 Mbps; its percentage is of that bound. The bottom row is what an unmodified stack delivers.

Sender	What sits above TCP	Goodput (Mbps)	of bound
erasure bound	n/a	≈ 78	100%
paced-UDP protocol [7]	its own reliability, no window	76.5	98%
iperf3	nothing, a raw stream	37.81	48%
scp	OpenSSH per-channel window	10.27–10.75	13%
scp + stock cubic	the same, plus cwnd halving and RTO backoff	0.07	0.09%

Contributions.

- **A quantified diagnosis.** Across a 66-cell $\text{RTT} \times \text{loss}$ grid, stock TCP tracks the Mathis $\sqrt{p}$ law without a fitted parameter at 5–10% loss and falls *below* it from 15% upward, where the sender is in timeout rather than in congestion avoidance. The arithmetic identifies window reduction, not the timer, as the dominant term (§4.2).
- **A twenty-line pluggable congestion control, no kernel rebuild,** that holds the window at the link’s BDP. It takes a complete 1 GiB scp from 0.07 to 10.75 Mbps and the grid from 12 to 61 of 66 cells above 10 Mbps, while reproducing stock cubic cell by cell on the lossless row (§4.3).
- **The hook, not the arithmetic, decides the outcome.** Identical pinning logic yields 3.5–4.8 Mbps from `cong_avoid()` and 9.57 Mbps from `cong_control()`, because under heavy loss the connection never leaves recovery and PRR owns the window instead (§4.4).
- **A negative result with a mechanism.** Removing the exponential backoff lowers the RTO peak in all six paired runs and stops it reaching Linux’s 120 s cap, yet moves no goodput, because the sender is `app_limited` in 20 of 24 samples. Measuring the same file under senders with no application window separates the layers: $\approx 4\times$ above the transport, $\approx 2\times$ inside it (§4.4, §4.5).

2 Related work

Loss as a congestion signal. Congestion collapse in the mid-1980s was solved by making every sender read loss as queue overflow and back off [2, 1]. Its steady-state consequence is the Mathis law [3]: a window-halving sender achieves at most $1.22 \text{ MSS} / (\text{RTT} \sqrt{p})$, independent of link rate. Where loss belongs to the medium the premise is false and the law describes a self-inflicted ceiling rather than a fair share. We use it as an unfitted prediction (§4.2) and as the argument for which mechanism to remove first (§3).

Removing the backoff. Mondal and Kuzmanovic [5] show that exponential backoff can be removed without endangering stability, provided endpoints keep the *implicit packet conservation principle*: a retransmission leaves only after the previous copy has left the network, which holds whenever the timeout exceeds the round trip. A congested gateway then serves each packet at most once per timeout whether or not later ones double, so backoff adds delay without reducing load. Our link meets their conditions in the extreme, which makes it a clean place to ask what the mechanism is worth; §4.4 finds that here it is worth nothing, for a reason their setting does not contain.

Decoupling the window from the loss signal. BBR [9] arrives from the other direction, setting its rate from estimated bottleneck bandwidth and propagation delay and treating loss as secondary; the kernel hook it uses [6] is the one our module turns out to need (§4.4). Where BBR estimates, we need not, since every link parameter here is known and the window can simply be a constant. Further out,

our own prior work on this testbed [7] abandons TCP for paced UDP with bitmap-driven negative acknowledgment and reaches 98% of the erasure bound. That is the top row of Table 1, and it is what lets §4.5 charge the residual gap to TCP’s loss recovery rather than to the link.

3 Method

3.1 Preliminaries: the two surfaces the diagnosis points at

The two loss responses live on different surfaces of the kernel, and the difference in cost shaped our approach.

Congestion control is a plugin interface, the one cubic and BBR are themselves written against, so a new algorithm is an out-of-tree module selected per host by a `sysctl`. Two of its hooks can set the window, and they are not interchangeable. `cong_avoid()` is consulted only when the core has not already decided, so whenever the flow is in recovery the kernel’s proportional rate reduction (PRR) owns the window and the hook is never reached. `cong_control()` replaces that machinery outright, and PRR never runs. The distinction is invisible at low loss and decisive at high loss. The retransmission timeout, by contrast, is core TCP compiled into the kernel image; changing it costs a full build, a bootloader edit and a reboot.

The arithmetic says which to do first. The Mathis ceiling at $p = 0.2$ and $\text{RTT} = 200 \text{ ms}$ is about 160 kbps, set by the window reduction and not by the timer, so a stack with a normal RTO but no backoff would still be two orders of magnitude below 10 Mbps. The cheap change is also the one that matters.

3.2 Modification (2): a congestion control that does not read loss as congestion

The module is about twenty lines of logic (`code/kmod/`). It never reduces the window on loss and holds it at the link’s bandwidth-delay product, 1730 segments. It registers *two* algorithms differing only in which hook does the pinning, `ee542` from `cong_avoid()` and `ee542c` from `cong_control()`, because §4.4 shows that choice decides the result. Taking over `cong_control()` also means taking over pacing, which the module must then set itself. Congestion control is a sender-side decision, so the module is loaded only on the client.

3.3 Modification (1): removing the exponential backoff

On an established connection, the retransmission timer normally doubles on each rearm; our patch instead recomputes it from the smoothed round-trip estimate, so the timeout tracks the path rather than the failure count. Three properties of that choice matter.

It is a `sysctl` rather than a hard-coded change, so both arms of every comparison in §4.4 run on one binary and one boot and the kernel version, configuration and compiler stop being confounds. The new branch is word for word the kernel’s existing thin-stream linear-timeout path: Linux already implements “do not back off”; it merely gates it on there being fewer than four packets in flight, which a sender holding 1730 segments can never satisfy. We attached existing logic to a new condition rather than inventing behaviour. And the connection can still give up, because the retry counter and its deadline live elsewhere in the stack. That deadline is derived assuming backoff, so without it the same wall-clock limit is reached after more retransmissions, but it is time-based and still fires. Connection setup is left untouched, since backoff during the handshake is what protects against SYN flooding. Build details are in Appendix B.

4 Experiments

4.1 Setup

Three AWS EC2 `c7i-flex.large` instances (2 vCPU, 4 GiB, Ubuntu 24.04) sit in one VPC: a client and a server on separate subnets, and a router with an interface in each, forwarding between them, which the TTL confirms. Construction and the failures we debugged are in Appendix B (Figure 7).

A token bucket sets the rate and a delay-and-loss qdisc the impairment, both on the router only. Where they sit matters twice over. The same loss applied on the sender’s own interface reads 54.3 Mbps where the router reads 31.4, because segmentation offload lets one drop remove a whole contiguous run that a single round of selective acknowledgment repairs; disabling offload recovers most of the difference (Appendix C). Delay placed on the sender differs as well, because the sender treats its own queue overflow as network congestion. And the bucket must be deep enough to hold a burst. A one-frame bucket drags a lossless 10 ms transfer down to 24.9 Mbps with the window near 18 segments, because TCP sees loss on a link that has none. The depth we use completes the same transfer in 91.0 s at 94.4 Mbps, the line rate. We measure how TCP responds to loss on the *link*, which requires the loss to fall outside the sender’s stack, per packet.

The workload is a 1 GiB file. Sweep cells run `scp` for a 60 s window, since a full transfer in the high-loss corner takes days, recording bytes landed on the server plus sender-side socket state every 5 s; headline transfers are complete and MD5-verified. The yardstick throughout is the erasure-channel capacity $(1 - p)R$: 78 Mbps at 20% loss, 100 Mbps at none. The 10 Mbps criterion is 13% of it.

4.2 Baselines: how far stock TCP is from the bound

Table 2: `scp` goodput under stock cubic in the 60 s window, selected cells, with the erasure bound for that loss rate and the fraction of it achieved. All 66 cells are in Figure 1. “0” means no payload byte reached the server in 60 s: at 20% loss the SSH key exchange alone is about ten round trips, each retried on loss.

RTT \ loss	0%	5%	10%	15%	20%
erasure bound (Mbps)	100	95	90	85	80
0 ms	95.4	26.0	2.61	0.56	0.10
20 ms	94.7	2.96	1.08	0.38	0.14
100 ms	91.7	0.66	0.28	0.07	0.10
200 ms	74.8	0.24	0.17	0	0
200 ms, of bound	75%	0.25%	0.19%	0%	0%

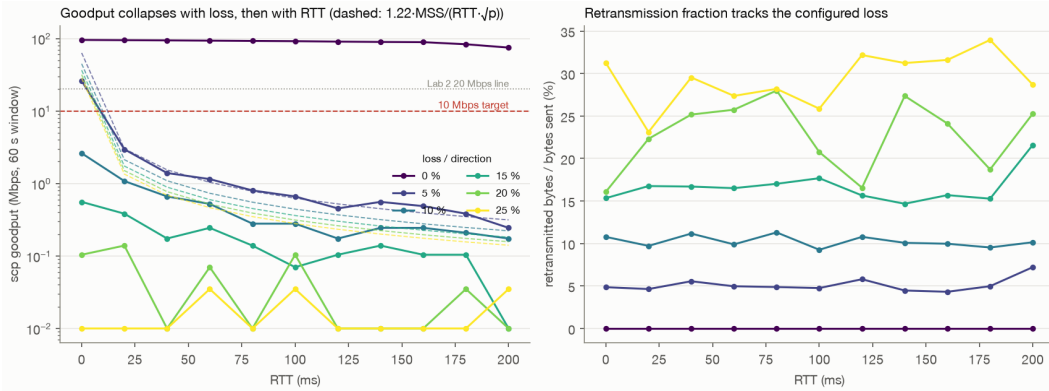


Figure 1: Left: goodput against RTT, one line per loss rate; dashed curves are the Mathis law with no fitted parameter. Right: the fraction of bytes retransmitted equals the configured loss at 5–10% and exceeds it at 20–25% (retransmissions of retransmissions, plus spurious timeouts). TCP is not losing more than the link drops; it is refusing to send.

Three readings come out of the grid (Table 2, Figures 1 and 2; the same grid as a heat map is Figure 9).

Loss, not delay, is the cliff. With no loss, `scp` holds 89–95 Mbps out to 160 ms and still 74.8 at 200 ms with zero retransmissions. Adding 5% loss at no added delay costs a factor of four; 20 ms of RTT on top costs another factor of nine.

The collapse follows the loss-rate model, until it does not. The dashed curves in Figure 1 are the Mathis law with no fitted parameter, and at 5–10% loss the measurements sit on them: 3.2 predicted against 3.0 Mbps at 5% / 20 ms, 0.32 against 0.24 at 5% / 200 ms, 0.22 against 0.17 at 10% / 200 ms.

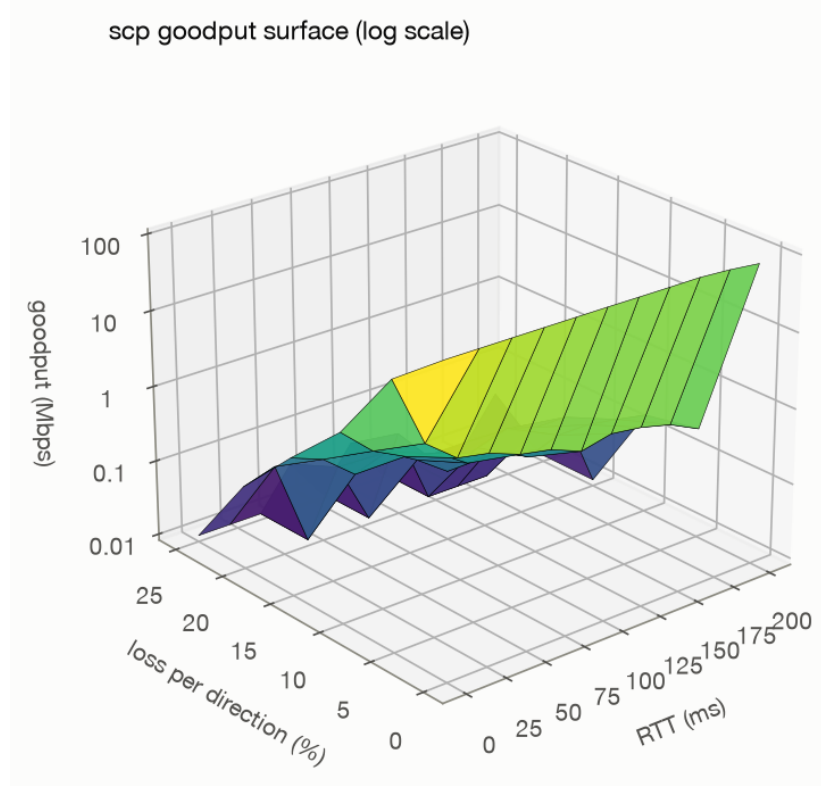


Figure 2: The same grid as a surface. Loss dominates: the surface falls by two orders of magnitude along the loss axis and by less than one along the RTT axis.

From 15% upward they fall *below* it, the sender then being in timeout rather than in congestion avoidance, which the model does not describe.

The target operating point is empty. At 200 ms / 20% nothing arrived in 60 s, and a full-length run of that cell measures a 23 kbps steady state. Ten Mbps is $400\times$ above that and $60\times$ above the Mathis ceiling of 0.16 Mbps for any window-halving sender, which rules out tuning as a route to it.

Figure 3 separates the two mechanisms: the window collapses to one or two segments, and the timer doubles to 0.4, 0.8, 1.6 s and eventually the kernel's 120 s cap. At 20% loss per direction a segment and its retransmission both die with probability 4%, which over 730,000 segments happens tens of thousands of times.

4.3 Main result: pinning the window

On the target link, a complete 1 GiB scp under ee542c finishes in 799.4 s, **10.75 Mbps**, MD5 verified; two further complete transfers, one with the backoff removal also enabled, give 800 s and 836.6 s, **10.7** and **10.27 Mbps**. Stock cubic on the same link moves 522 kB in a 60 s window, 0.07 Mbps. Over those 799 seconds, 78 of the 79 sender-side ss -tin samples, taken every 10 s, report cwnd: 1730; the exception, 290 s in, catches the sender in a timeout with the window at 1 after three backoffs. The RTO sits between 403 and 923 ms in 75 of 79 samples and peaks at 4.1 s, far from the 120 s cap, and bytes_retrans/bytes_acked settles at 26%, which is what 20% link loss plus the 4% of retransmissions that are themselves lost predicts. The same cell measured under the 60 s cap used elsewhere reads 9.57 Mbps. We attribute the difference to SSH handshake and scp setup, which are expensive at this loss rate and amortised over a full transfer.

Rerunning the full 66-cell sweep under identical shaping (Figures 4, 5) takes the cells above 10 Mbps from **12 of 66** to **61 of 66**; at 200 ms the progression with loss is 74.5, 26.3, 19.3, 14.5, 9.6 and 5.3 Mbps against cubic's 74.8, 0.24, 0.17, 0.00, 0.00 and 0.04. The control is the lossless row, 95.4

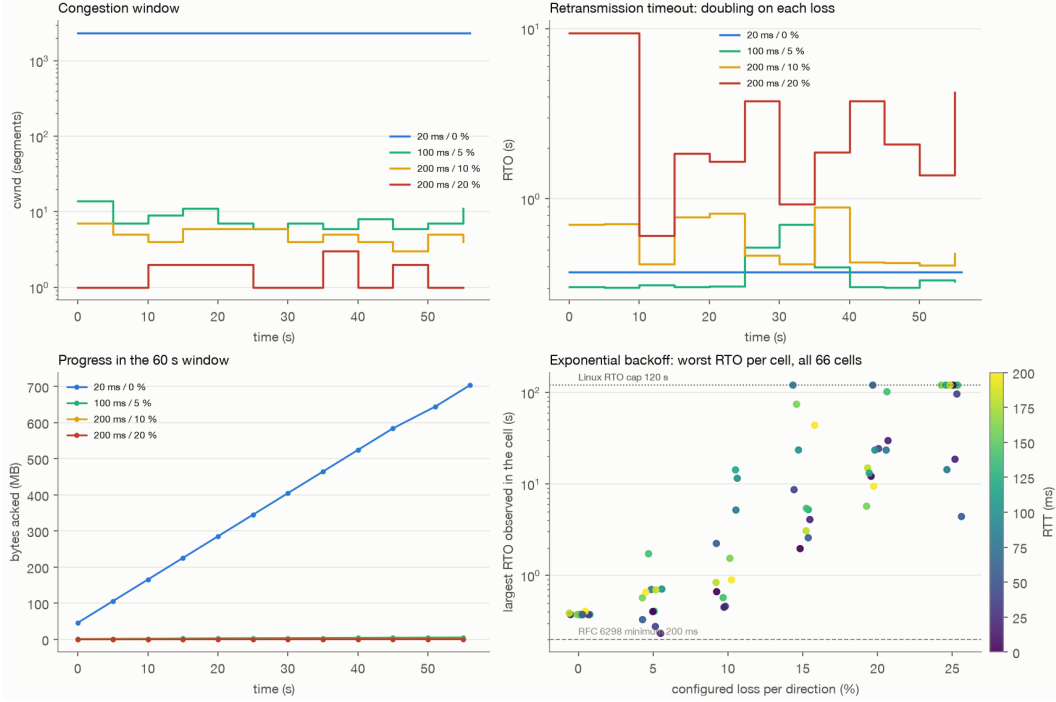


Figure 3: Sender-side TCP state from `ss -tin`, sampled every 5 s. Top left: with no loss (20 ms) the window sits near 2300 segments; with loss it collapses to 1–10 and at 20% is pinned at 1–2. Top right: the timeout doubles on consecutive losses; at 200 ms / 20% it swings between 1 and 10 s. Bottom left: bytes acknowledged in the window. Bottom right: the largest RTO seen in each of the 66 cells against configured loss; from 15% upward the timer reaches Linux’s 120 s cap.

against 95.4, 91.7 against 91.8, 74.8 against 74.5, cell by cell across all eleven RTTs: a controller that refuses to read loss as congestion should change nothing on a link that has none, and it does not.

4.4 Ablations

The hook is worth a factor of two. At 200 ms / 20%, `ee542` gives 3.52, 4.56, 3.59 and 4.77 Mbps over four runs and ends with windows of 14, 2, 1 and 1 segments; `ee542c` gives 9.57 Mbps and ends at 1730. The pinning is written identically in both. At 20% loss the connection is in recovery almost continuously, so in the `cong_avoid` variant PRR sets the window on nearly every acknowledgment and the module’s hook is never reached. The call path, not the arithmetic inside it, moves the number.

Table 3: Each modification alone and combined, at 100 Mbps with 200 ms RTT and 20% loss each way. Three repetitions per cell of a 120 s capped `scp`; the three numbers in each entry are the three runs. The two columns differ only by `sysctl net.ipv4.tcp_no_rto_backoff`, on one kernel binary and one boot. “RTO peak” is the largest `rto` in that run’s `ss` samples.

	goodput (Mbps)		RTO peak (s)	
	backoff on	backoff removed	backoff on	backoff removed
<code>cubic</code>	0.052 / 0.017 / 0.087	0.035 / 0.052 / 0.070	31.7 / 120.0 / 120.0	21.0 / 7.0 / 3.2
<code>ee542c</code>	14.29 / 11.82 / 10.53	14.00 / 13.77 / 9.92	1.24 / 3.41 / 0.80	0.53 / 0.76 / 0.77

Backoff removal is visible in the timer. The RTO peak is lower with backoff removed in *all six* paired runs (Table 3, Figure 6). The clearest pair is `cubic`, where two of the three baseline runs drove the timer to 120 s, Linux’s `TCP_RTO_MAX` cap, while the same configuration with backoff removed peaked at 21.0, 7.0 and 3.2 s. Under `ee542c` the effect is smaller in absolute terms but just

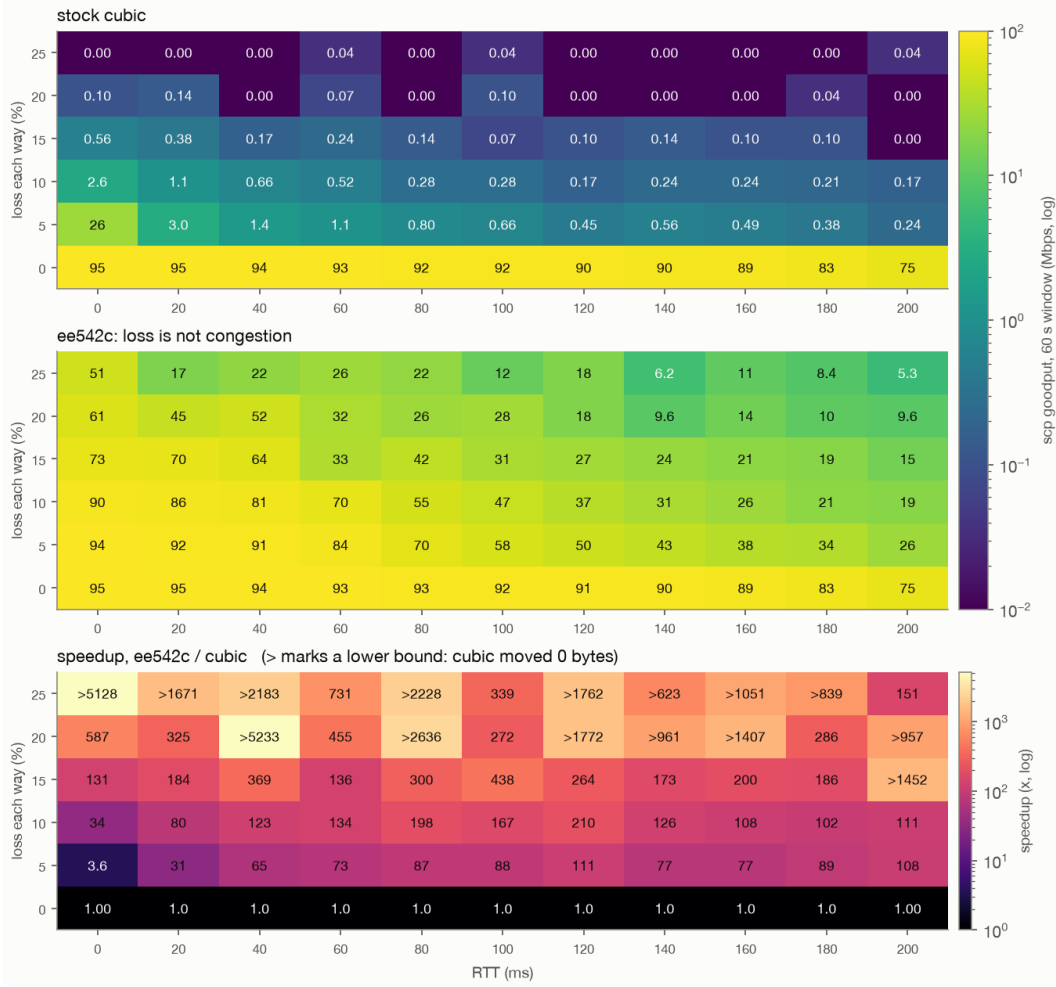


Figure 4: The same RTT $\times$ loss grid under stock cubic (top) and under ee542c (middle), on one log colour scale, with the cell-by-cell speedup below. Cells marked “>” are lower bounds: cubic landed no measurable data there in 60 s. The bottom row of every panel is the control: with no loss the two stacks agree cell by cell.

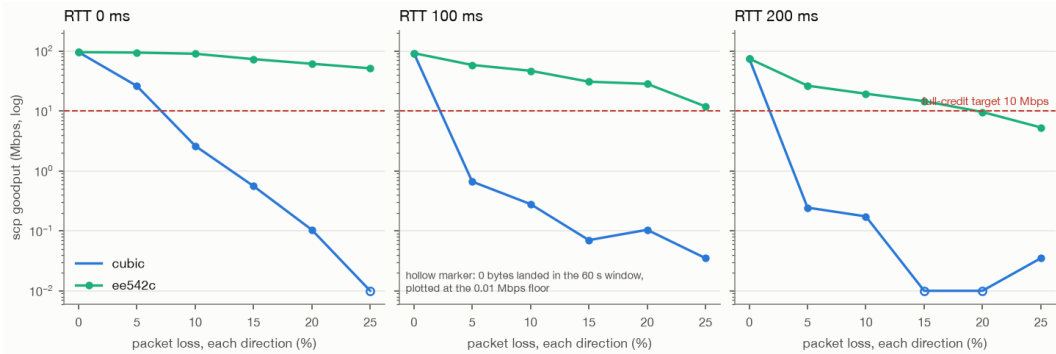


Figure 5: Goodput against loss at three round-trip times. The gap opens as soon as any loss exists and is two to three orders of magnitude by 15%. Hollow markers are cells where cubic moved zero bytes in the window, drawn at the 0.01 Mbps plotting floor.

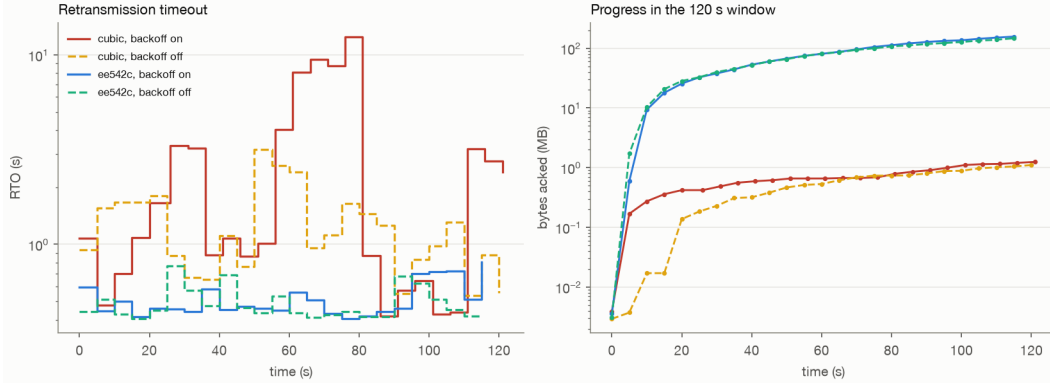


Figure 6: One repetition of each cell of Table 3. Left: the retransmission timer. With backoff on, cubic climbs to 12 s and beyond; with it removed, the timer tracks the RTT estimate. Under ee542c it stays near 0.5 s either way. Right: the same four runs differ almost not at all in progress, which is the subject of §4.5.

as consistent, 1.24/3.41/0.80 s against 0.53/0.76/0.77 s: the timer never leaves the neighbourhood of the RTT estimate.

It buys no throughput at this operating point. Goodput is unchanged within run-to-run scatter: cubic averages 0.052 Mbps either way and ee542c 12.21 against 12.56, both differences smaller than the spread between repetitions of one cell. The mechanism is layered. What pins cubic to the floor is the window, not the timer, so modification (1) alone cannot move it; and once modification (2) has released the window, the binding constraint is no longer in the transport at all.

4.5 Analysis: where the remaining capacity goes

The kernel answers directly. In **20 of 24** samples the ee542c sender is flagged `app_limited`, which the kernel sets when the controller is consulted and the socket has nothing left to send. Receive window accounts for 1.4% of the time, the congestion window never leaves 1730, and the path is not the constraint either, its measured delivery rate peaking at 65.5 Mbps. The sender is neither window-limited nor timer-limited; it is starved from above.

The feeder is `scp`, and OpenSSH runs its own per-channel flow control on top of TCP, advancing credit by explicit messages that must make the round trip. On a 200 ms path dropping one segment in five, that loop is far slower than TCP’s own, and TCP idles whatever its window says.

Rather than leave that an inference we measured it, moving the same file across the same link and kernel under senders differing only in what sits above TCP (Table 1). A raw TCP stream, with no application-level window at all, reaches **37.81 Mbps**. That is 3.5–3.7× the three complete `scp` transfers, on an unchanged kernel.

Yet the raw stream still reaches only 48% of the erasure bound where our paced-UDP protocol [7] reaches 98% of it on the same link and MTU. The missing capacity is therefore not all above the transport: at 20% loss, selective-acknowledgment recovery, the reordering it induces and the 21% of bytes spent retransmitting are a real transport-layer cost that a protocol built for a known-lossy link does not pay. Roughly a factor of four of application overhead, then a factor of two of TCP’s own loss recovery.

5 Discussion and limitations

The margin is thin. Three complete transfers gave 10.75, 10.7 and 10.27 Mbps against a 10 Mbps criterion, a worst-case margin of 2.7%, and 120 s capped measurements of the same cell scatter from 9.92 to 14.29 Mbps. Any claim at this cell is a full-transfer average, not a point estimate.

The window is a constant, which is only safe here. ee542c holds 1730 segments because the bandwidth-delay product is known and the link is not shared. On a contended path the same module invites congestion collapse: it never yields, so it would take the whole bottleneck from any conforming flow. Nothing here argues that loss should be ignored in general, only that it carries no congestion information on a link whose loss is a property of the medium, and we provide no mechanism to establish that precondition. Five of 66 cells also remain below 10 Mbps, at the highest loss rates and longest delays; 25% loss at 200 ms reaches 5.3 Mbps.

The backoff null result is operating-point specific. At a cell where the application is not the binding constraint, modification (1) could well matter, and we did not sweep for one. More broadly, every measurement is a single unidirectional flow with synthetic netem loss that is independent per packet, whereas real lossy links are bursty and real paths carry cross traffic; both could change the recovery behaviour that §4.5 attributes a factor of two to.

6 Conclusion

Loss on this link carries no congestion information, so both of TCP's responses to it are unearned. Removing the first, with twenty lines of pluggable congestion control and no kernel rebuild, takes a complete 1 GiB transfer from 0.07 to 10.75 Mbps and lifts 61 of 66 grid cells above 10 Mbps while leaving the lossless row untouched. The Mathis law predicted that the window and not the timer was the dominant term. Identical arithmetic in the wrong hook still costs a factor of two, so reading the call path mattered more than designing the algorithm. Removing the second changes the timer in all six paired runs and leaves goodput within run-to-run scatter, because by then the sender is app_limited in 20 of 24 samples and the constraint has moved into scp's channel window. Two things follow. A correct kernel modification can be worth nothing at a particular operating point, and the only way to find out is to build it and measure. And the capacity still missing at 37.8 Mbps, where the application is gone entirely, is TCP's own loss recovery, which is the part no configuration change reaches.

Code and data. Everything in this report is reproducible from <https://github.com/TINKPA/ee542-group6> under lab3/: the testbed provisioning scripts, the congestion-control module, the kernel patch, the measurement drivers, and the raw samples behind every figure. Each member worked on their own branch and merged to main, so the repository's history also records who did what.

A Handout questions

The body is organised as an argument rather than as a walk through the assignment. This appendix answers each question in order, in at most three sentences, pointing into the body where the answer is developed there.

Why must Source/Destination check be disabled on the router's interfaces? Every interface carries a hypervisor-level filter dropping packets whose source or destination is not one of its own private addresses, which is correct for a host and fatal for a router, since a router forwards packets addressed to *other* hosts. With the check on, every forwarded packet is silently discarded: ping shows 100% loss while tcpdump on the router sees the packet arrive and its forwarded copy never leave.

How is Amazon able to convert your public IP and reach your private IP on the interface? An Elastic IP is never configured inside the guest, which sees only its private address. It is a one-to-one NAT entry in the Internet Gateway, rewriting destination on ingress and source on egress, keyed on the interface's *private* address.

Why does SSH go down if the IP on that interface is changed? Because of that keying: change the private address and the NAT entry points at nothing. The session cannot even be recovered, since the recovery path rides on the same mapping, which is why management lives on an interface no experiment touches.

What do AMI, instance type and IAM mean here? An AMI is a root-volume snapshot plus launch metadata, resolved through a public parameter rather than a hard-coded ID so it never goes stale. The instance type fixes vCPU, memory and network ceiling, and ours runs on a newer virtualisation platform than the assignment's, which matters for the `ethtool` question below. IAM is the permission layer: the root user only ever creates the CLI user.

Why can't the VM reach the Internet, and what does `resolv.conf` have to do with it? Two independent things must hold, a default route out through the Internet Gateway with a public address attached, and DNS; `resolv.conf` is only the second. Ubuntu 16.04 needs the DHCP-supplied resolver written in by hand where 24.04 picks it up itself, so on our nodes the failure is always the routing half: on the four-subnet topology the default route sits on the interface with no Elastic IP, so packets leave through a gateway holding no NAT entry for them.

Why two NICs per endpoint? Because the tutorial shapes the endpoint's own interface, and if SSH rode that interface, shaping the experiment would shape the control channel. A second interface in its own subnet keeps management out of band and untouched.

Why do the UDP and TCP readings differ? UDP is sent at the *offered* rate and the tool reports what arrived, whereas TCP reports what its congestion controller was willing to send. Behind a 100 Mbps bucket TCP reads 86–95 Mbps and UDP 97 with 2.8% loss, that loss being the tail drop of the offered excess.

add **versus** change. add installs a new root qdisc and fails if one is already there; change edits the existing one in place, which is why the sequence switches to it once a qdisc exists.

What changes when delay and loss are added, and why? Adding 100 ms leaves the rate untouched but drops TCP from 5.5 Gbit/s to 129 Mbps, and the cause is not the window, which peaked large enough to carry 370 Mbps at the measured 101 ms. It is the delay node's own 1000-packet queue: 1000 frames held for 101 ms is 119 Mbps, and the same limit explains the 42% UDP loss in a step that configures no loss at all. A delay node is silently also a loss node whenever offered load $\times$ delay exceeds its limit; raising the MTU confirms this by making the phantom loss disappear. Configured loss then behaves as asked: 10% costs UDP exactly 10% and drops TCP to 8.4 Mbps.

What does `tbdf` do? Tokens accrue at the configured rate and a packet leaves only if the bucket holds its size in tokens, so `burst` is the bucket depth and `latency` bounds how long a packet may wait. At 100 Mbps it caps both protocols at the line rate. Drive `latency` to near zero and the queue disappears, so the shaper becomes a loss source for any sender whose bursts exceed the depth (§4.1).

What does changing the MTU do, and what appears in `dmesg`? Nothing appears in the log under either qdisc or on an MTU change, since the adapter accepts anything up to the VPC's 9001-byte jumbo limit. The MTU effects are all in the shaper: a one-frame bucket holds exactly one jumbo frame, which drops TCP to 6 Mbps while paced UDP is unaffected.

What does `ethtool -s eth0 speed 10` return, and why does AWS not allow it? Operation not supported. Link speed is a property of the virtual fabric, enforced per instance type by the hypervisor rather than negotiated by a physical layer the guest owns, so the driver exposes no speed setter and rate limiting must be done in software.

How is `dhclient` started on an interface in VyOS? From the documentation rather than from our own run, since we could not launch it: the interface is given address `dhcp` and committed, and a lease is renewed in operational mode. The assignment's VyOS image has been deregistered, and every VyOS image still published in the region is a paid Marketplace product, which a dry run confirms by refusing with `OptInRequired` where Ubuntu succeeds. We therefore use an Ubuntu router with forwarding enabled, functionally the same middlebox.

Do you get the same observations with the shaper on the router's non-EIP interface? No, and systematically so: rate limiting is location-independent but delay and loss are not. Developed in §4.1, with the three-arm comparison in Appendix C.

Baseline throughput, the $RTT \times$ loss sweep, and the evaluation of each modification. §4.1 (baseline, 94.4 Mbps at 10 ms with no loss), §4.2 (the 66-cell sweep), §4.3 (the sweep rerun under the modified stack) and §4.4 (each modification alone and combined).

Porting the transfer program to the cloud and comparing with the earlier results. Appendix D.

B Testbed construction and operational notes

Topology. A VPC 10.200.0.0/16 holds two subnets, 10.200.1.0/24 (client 10.200.1.83) and 10.200.2.0/24 (server 10.200.2.48), in one availability zone. An Internet Gateway is attached and each subnet’s route table carries 0.0.0.0/0 $\rightarrow$ igw. The router is an Ubuntu 24.04 instance with one ENI in each subnet, `ip_forward=1`, and an Elastic IP on its first ENI. Every VPC route table has an immutable `local` route for the whole VPC CIDR, so by default the two subnets talk directly and a router instance would be bypassed; AWS resolves routes by longest prefix match, so we add 10.200.2.0/24 $\rightarrow$ eni-A to subnet A’s table and 10.200.1.0/24 $\rightarrow$ eni-B to subnet B’s, and a /24 beats the /16. The proof is the TTL: ping from client to server returns `ttl=63`, and `tcpdump -ni any icmp` on the router shows the echo request entering on `enp39s0` and leaving on `enp40s0`. Built with `code/aws/aws_lab.sh`.

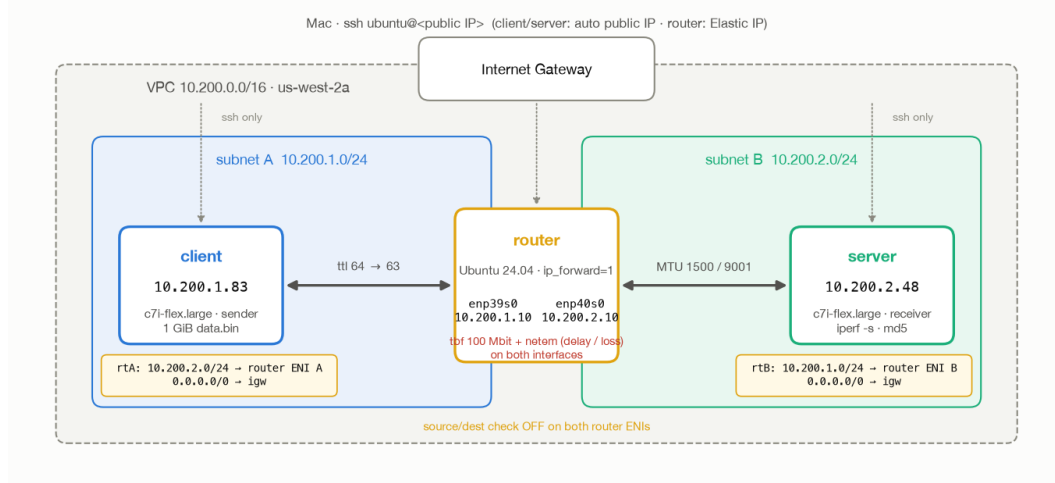


Figure 7: The testbed. Three EC2 instances in one VPC; per-subnet route tables carry a more-specific route for the other subnet pointing at the router’s interface in this subnet, so client–server traffic cannot take the VPC’s implicit `local` route. Shaping lives on the router only.

Issues debugged. Four failures cost time and none changed the design. A new account’s 5-vCPU quota does not cover three 2-vCPU nodes (raised to 32 in four minutes via L-1216C47A); `t3.medium` fails with `InvalidParameterCombination` on a free-plan account, while `c7i-flex.large` is eligible and faster; the router’s second ENI comes up unaddressed, and on Nitro it is named `enp40s0` rather than the `ens6` of most tutorials, so the setup script discovers interfaces live and writes a static netplan file. The instructive one is that a `stop/start` cycle broke forwarding while `ip_forward` and the VPC routes stayed intact. `tcpdump` showed each echo request leaving on the interface it arrived on, with an ICMP redirect. Cloud-init had re-rendered its config with DHCP on the second ENI, and the lease had replaced the connected route with a lower-priority default, so the kernel forwarded toward the VPC gateway, which dropped the packet. `dhcp4: false` in the netplan override fixes it.

Building the kernel. A full build on a small instance is budgeted at ten hours. We built incrementally on a disposable host from the source tree matching the running kernel, and added one step: trimming the configuration against the *target node*’s loaded module list. That builds 48 modules rather than several thousand and finishes in **31 min 26 s** on two CPUs. Built-in options are left untouched, so the network and storage drivers survive and the result is bootable. We booted the disposable host on

it before touching the client, since an unbootable cloud instance can only be deleted. The installed kernel identifies itself as 7.0.14-ee542.

That trimming has one trap worth recording, because it cost us a measurement pass. The socket-statistics module is built into stock Ubuntu kernels and never appears as a loaded module, so the trimming drops it. Nothing fails loudly: `ss` silently falls back to the legacy `/proc` interface, which carries no byte counters and no RTT, and reports the timeout in the wrong unit. Every sender-side number in this report comes from `ss`, so a configuration choice had quietly downgraded the instrument. The rule generalises: such trimming sees only what was loaded as a module, so anything built in, observability above all, must be re-enabled by name. The corrected rebuild was incremental and took **1 min 30 s**.

C Where the shaper belongs

The two-subnet testbed of §4.1 has one NIC per endpoint, which is enough for the experiments but does not match the tutorial. We therefore built a second, independent testbed (code/aws/lab3_topo.sh, VPC 10.0.0.0/16) with four subnets: 10.0.1.0/24 and 10.0.2.0/24 carry the client–router and router–server legs, and 10.0.3.0/24 and 10.0.4.0/24 exist only so that the client and server each get a second ENI with an Elastic IP for SSH. Forwarding is proved the same way, `ttl=63` from client to server.

On it we ran the tutorial’s `tc` sequence at three places, each impairing only the client→server direction so the arms are comparable, at the native MTU of 9001 (Table 4).

Table 4: The same `tc` steps applied at three places. `iperf3` TCP for 10 s then UDP for 10 s offered at 200 Mbit, MTU 9001. “endpoint” is the client’s experiment NIC, “router” its non-EIP leg; the third arm repeats the first with GSO and GRO disabled.

tc step	endpoint	router	endpoint, no GSO/GRO
no qdisc	9518.5 Mbps	9516.9	9518.8
netem delay 100ms	143.2 (218 retx)	209.3 (0 retx)	117.7 (122 retx)
netem loss 10%	54.3	31.4	33.9
tbfb 100mbit burst 9015	96.7	97.9	96.4
UDP loss, loss 10% step	9.85%	10.08%	9.90%
UDP loss, tbfb step	50.30%	50.88%	50.30%

Three readings. Stateless impairment is location-independent, the unshaped baseline agreeing to within 0.02% and the token bucket to within 1.5%, because a bucket is a bucket wherever it stands. Delay is not: all three arms measure the same RTT, but the endpoint arms retransmit 218 and 122 times against the router’s 0, because on the sender the delay queue is also a loss source and TCP reads its own overflow as network congestion. Loss is not either, and the reason is aggregation: with offload on, one drop removes a whole multi-segment super-packet that selective acknowledgment repairs in a single round, whereas on the router the packets are already split and 10% falls per packet, which is far harsher. Disabling offload moves the endpoint from 54.3 to 33.9 Mbps, close to the router’s 31.4. The UDP columns are the control: with no feedback loop they reproduce the configured rate in every arm, placing the whole discrepancy in TCP’s control loop rather than in the impairment.

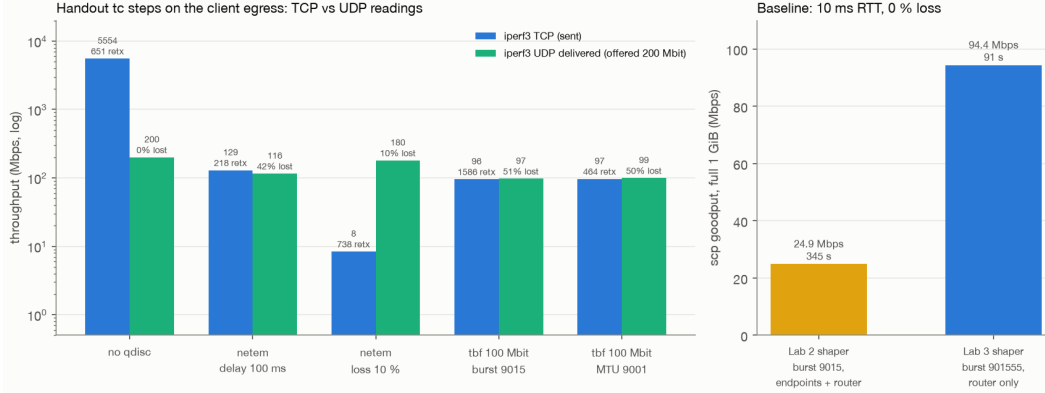


Figure 8: Left: TCP and UDP readings through the tutorial’s `tc` steps on the client egress (log scale). Right: a full 1 GiB `scp` at 10 ms with no loss under a one-frame bucket (burst 9015, endpoints and router) and under the bucket used throughout the body (burst 901555, router only).

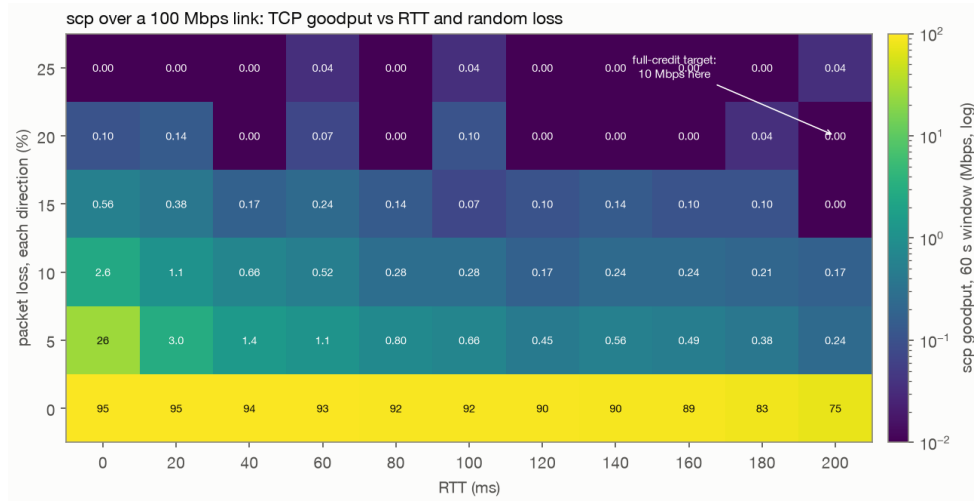


Figure 9: The stock-cubic grid of §4.2 as a heat map (log colour scale). The operating point studied in §4.3, 200 ms and 20%, is the top-right corner.

D The paced-UDP protocol on this testbed

We ported our prior protocol [7] (paced UDP data, bitmap-driven NAK retransmission, TCP only for session setup) to the EC2 nodes unchanged, the same binary built with the same `Makefile`, and reran its matrix: three link cases $\times$ two MTUs $\times$ three repetitions, every run MD5-verified (Table 5, Figure 10).

Two observations carry into the body. The cloud is a *cleaner* testbed than either local one: measured effective loss equals the configured `netem` loss to within 0.7%, whereas the emulated VMs added 5–15% of their own. And the streaming control plane, which never builds a long negative-acknowledgment list, sits within 2% of the bound in Case 2 at either MTU: 76.5 Mbps against 78.1 at MTU 1500 and 77.8 against 79.7 at MTU 9001. The MTU-1500 figure is the one used as the top reference row of Table 1, since every other row of that table was also measured at MTU 1500.

Table 5: One-way time for 1 GiB (median of 3, MD5 verified in every run). The UTM testbed cannot carry jumbo frames (vmnet rejects MTU > 1500), which is why the earlier work used a netns/veth replica for MTU 9000. The bound is $F/((1-p)\eta R)$ with payload fraction $\eta = 1464/1500$ at MTU 1500 and 8965/9001 at MTU 9001 [7], so it depends on the MTU. The AWS rows report the faster of the protocol’s two control planes, the streaming sender; the local testbeds predate it and ran only the round-based one, which on AWS reads 124.8 s (68.8 Mbps) in Case 2 at MTU 1500, its deficit being a burst of negative acknowledgments dropped by the receiver’s own shaper rather than a data-path cost [7].

Testbed	Case 1 (10 ms / 1%)	Case 2 (200 ms / 20%)	Case 3 (200 ms / 0% / 80 M)
UTM VMs, x86 emulated, MTU 1500	115.3 s (74.5 Mbps)	134.9 s (63.7)	124.5 s (69.0)
netns + veth, MTU 1500	109.5 s (78.4)	136.8 s (62.8)	125.3 s (68.6)
AWS EC2, MTU 1500	89.8 s (95.6)	112.3 s (76.5)	111.3 s (77.2)
netns + veth, MTU 9000	161.2 s (53.3)	206.6 s (41.6)	171.6 s (50.1)
AWS EC2, MTU 9001	88.6 s (96.9)	110.5 s (77.8)	111.9 s (76.8)
erasure bound, MTU 1500	88.9 s	110.0 s	110.0 s
erasure bound, MTU 9001	87.1 s	107.8 s	107.8 s
scp on AWS, MTU 1500	548 s (15.7)	≈4 d (23 kbps)	343 s (25.0)

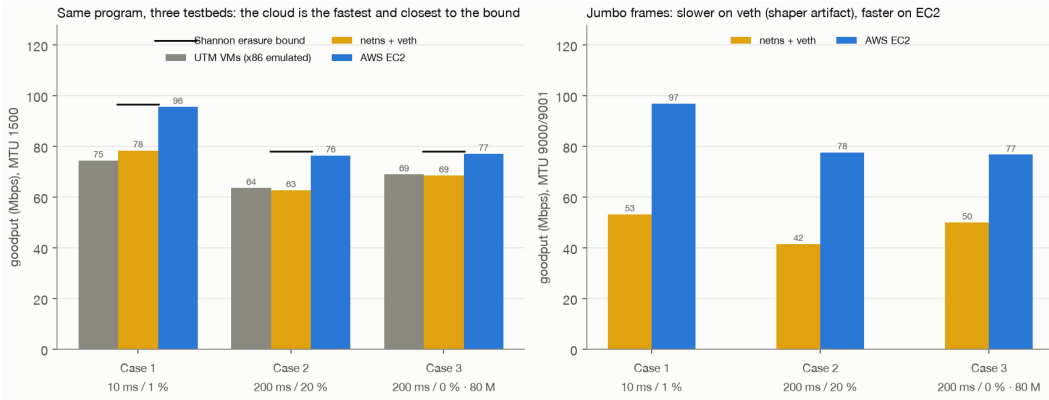


Figure 10: The same program on three testbeds. Left: at MTU 1500 the cloud is fastest in every case and within 1.2% of the bound in Cases 1 and 3. Right: jumbo frames are much slower on veth, a shaper artifact rather than a property of the protocol, and faster on EC2 in Cases 1 and 2; Case 3 is the one cell where they are marginally slower, the streaming sender’s pacing jitter costing it a dropped frame now and then against a one-frame bucket [7].

References

- [1] V. Jacobson, “Congestion avoidance and control,” in *Proc. ACM SIGCOMM*, 1988, pp. 314–329.
- [2] J. Nagle, “Congestion control in IP/TCP internetworks,” *ACM SIGCOMM CCR*, vol. 14, no. 4, pp. 11–17, 1984.
- [3] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, “The macroscopic behavior of the TCP congestion avoidance algorithm,” *ACM SIGCOMM CCR*, vol. 27, no. 3, pp. 67–82, 1997.
- [4] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s retransmission timer,” RFC 6298, 2011.
- [5] A. Mondal and A. Kuzmanovic, “Removing exponential backoff from TCP,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 5, pp. 19–28, Oct. 2008.
- [6] M. Pfeiffer, “Pluggable TCP congestion control algorithms in the Linux kernel,” technical report, 2017.
- [7] H. Tang, X. Yang, and J. Lu, “Reliable transfer on lossy, high-latency links: round-based and streaming NAK retransmission over paced UDP,” EE542 Lab 2 report, University of Southern California, 2026.

- [8] Amazon Web Services, “Route tables for your VPC” and “Elastic IP addresses,” *Amazon VPC / EC2 User Guides*, 2026.
- [9] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, “BBR: Congestion-based congestion control,” *Communications of the ACM*, vol. 60, no. 2, pp. 58–66, 2017.